```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np
```

```
sesh_df = pd.read_csv("/content/drive/MyDrive/2024-25/WUDAC Datathon/sessions.csv")
```

```
# del trans_df

trans_df = pd.read_csv("/content/drive/MyDrive/2024-25/WUDAC Datathon/transactions.csv")
```

```
trans_df.describe(include='all')
```

	machine_id	site_session_id	prod_category_id	prod_name	domain_id	prod_qty	prod_totprice	basket_tot	eve
count	2.308972e+06	2.308972e+06	2.308972e+06	2308972	2.308972e+06	2.299872e+06	2.308945e+06	2.281021e+06	
unique	NaN	NaN	NaN	1674423	NaN	NaN	NaN	NaN	
top	NaN	NaN	NaN	1297519000000009471 3OX1IS75JV0I	NaN	NaN	NaN	NaN	20
freq	NaN	NaN	NaN	1681	NaN	NaN	NaN	NaN	
mean	3.097972e+08	1.841756e+18	5.104869e+09	NaN	5.047765e+18	1.430984e+00	6.694491e+01	9.511895e+01	
std	2.480608e+07	2.817583e+18	2.712404e+09	NaN	2.521380e+18	2.795970e+02	5.204796e+04	3.003336e+02	
min	9.567401e+07	0.000000e+00	9.990000e+02	NaN	3.853290e+18	1.000000e+00	0.000000e+00	0.000000e+00	
25%	2.905126e+08	0.000000e+00	3.004000e+09	NaN	4.046670e+18	1.000000e+00	5.980000e+00	1.695000e+01	
50%	3.098154e+08	0.000000e+00	7.008023e+09	NaN	4.046670e+18	1.000000e+00	1.400000e+01	4.799000e+01	
75%	3.289923e+08	3.433591e+18	7.008026e+09	NaN	4.046670e+18	1.000000e+00	2.800000e+01	1.110900e+02	
max	3.715724e+08	9.223348e+18	1.001605e+10	NaN	1.277397e+19	4.240000e+05	7.906183e+07	3.516580e+05	

```
sesh_df.describe(include='all')
```

	machine_id	site_session_id	user_session_id	domain_id	ref_domain__name	pages_viewed	duration	event_date	event_tim
count	1.376567e+07	1.376567e+07	1.376567e+07	1.376567e+07	3546162	1.373791e+07	1.373399e+07	13751991	1374474
unique	NaN	NaN	NaN	NaN	10653	NaN	NaN	29325	8640
top	NaN	NaN	NaN	NaN	amazon.com	NaN	NaN	2020-11-30	0:00:0
freq	NaN	NaN	NaN	NaN	1602311	NaN	NaN	21666	227
mean	3.078465e+08	4.611823e+18	2.599448e+15	6.068540e+18	NaN	9.539289e+00	1.110329e+01	NaN	Na
std	2.558799e+07	2.662835e+18	1.272628e+14	3.217254e+18	NaN	2.142559e+01	2.869295e+01	NaN	Na
min	5.438873e+07	5.668573e+11	2.657689e+14	3.853290e+18	NaN	1.000000e+00	-7.000000e+00	NaN	Na
25%	2.893530e+08	2.305250e+18	2.502640e+15	4.046670e+18	NaN	2.000000e+00	1.000000e+00	NaN	Na
50%	3.073618e+08	4.612783e+18	2.573338e+15	4.046670e+18	NaN	4.000000e+00	2.000000e+00	NaN	Na
75%	3.277368e+08	6.918310e+18	2.691783e+15	1.058424e+19	NaN	1.000000e+01	1.200000e+01	NaN	Na
max	3.715804e+08	9.223372e+18	2.883716e+15	1.277397e+19	NaN	6.963000e+03	1.440000e+03	NaN	Na

```
sesh_df
```

	machine_id	site_session_id	user_session_id	domain_id	ref_domain__name	pages_viewed	duration	event_date	event_t
0	288084937	6729633401426801739	2412880849370001	1.058424e+19	NaN	14.0	14.0	2020-01-01	0:0
1	288912140	1092751701181111355	2412889121400001	1.058424e+19	NaN	34.0	31.0	2020-01-01	0:0
2	285811167	7837313750729559995	2412858111670001	4.046670e+18	NaN	4.0	3.0	2020-01-01	0:0
3	290119393	192904185306363414	2412901193930001	4.046670e+18	amazon.com	15.0	9.0	2020-01-01	0:0
4	306223388	6798383743536217771	2413062233880001	4.046670e+18	NaN	23.0	53.0	2020-01-01	0:0
...	...	...	...	...	...	...	...	...	
13765661	311301658	2640888765140691712	2653113016580035	4.046670e+18	NaN	4.0	1.0	NaN	N
13765662	318185954	712868840266094016	2653181859540054	4.046670e+18	imdb.com	43.0	62.0	NaN	N
13765663	322449434	5983590363902492002	2683224494340041	4.046670e+18	NaN	5.0	1.0	NaN	N
13765664	311314506	4677707501469914285	2833113145060027	4.046670e+18	NaN	2.0	0.0	NaN	N
13765665	286181354	4467295126475715506	2632861813540129	4.046670e+18	amazon-adsystem.com	1.0	1.0	NaN	N

13765666 rows × 20 columns

```
sesh_df[(sesh_df['event_date'] > '2022-01-01') & (sesh_df['event_date'] < '2024-01-01')]
```

	machine_id	site_session_id	user_session_id	domain_id	ref_domain__name	pages_viewed	duration	event_date	event_t
9462871	320932659	1825149789255760809	2653209326590002	4.046670e+18	NaN	18.0	23.0	2022-01-02	0:0
9462872	328142529	5058232221636859853	2653281425290001	3.853290e+18	bing.com	17.0	13.0	2022-01-02	0:0
9462873	309164329	6378582037696062072	2653091643290003	4.046670e+18	amazon.com	13.0	12.0	2022-01-02	0:0
9462874	320821539	2795475114685183286	2653208215390003	4.046670e+18	amazon.com	1.0	1.0	2022-01-02	0:0
9462875	336141183	7288771008337791205	2653361411830008	4.046670e+18	NaN	5.0	1.0	2022-01-02	0:0
...	...	...	...	...	...	...	...	...	
13751950	353556721	1279357329715199341	2883535567210059	1.058424e+19	google.com	1.0	1.0	2023-12-31	N
13751951	363780116	895044726984726721	2883637801160098	1.277397e+19	NaN	1.0	1.0	2023-12-31	N
13751952	284943833	4380148603803325793	2882849438330057	4.046670e+18	NaN	1.0	1.0	2023-12-31	N
13751953	305491320	790750504438825492	2883054913200096	4.046670e+18	amazon.com	42.0	29.0	2023-12-31	N
13751954	368732447	2321075934243236729	2883687324470043	4.046670e+18	NaN	1.0	1.0	2023-12-31	N

4274125 rows × 20 columns

```
len(sesh_df[(sesh_df['event_date'] > '2022-01-01') & (sesh_df['event_date'] < '2024-01-01')]['site_session_id'].unique())
```

```
4231790
```

```
trans_df[(trans_df['event_date'] > '2022-01-01') & (trans_df['event_date'] < '2024-01-01')]
```

	machine_id	site_session_id	prod_category_id	prod_name	domain_id	prod_qty	prod_totprice	basket_tot	event_date	ev
1517226	319978077	0	7009032145	4pcs Set Oven Mitts and Pot Holders Oven Glove...	1.058424e+19	1.0	24.56	14.22	2022-01-02	
1517227	319978077	0	7009031127	5pack Cotton Pocket Pot Holder Set Kitchen Hea...	1.058424e+19	1.0	23.68	14.22	2022-01-02	
1517228	319978077	0	7009032145	Oven Mitts and Pot Holders 4pcs Set Oven Glove...	1.058424e+19	1.0	12.99	14.22	2022-01-02	
1517229	345943601	0	7008024000	Sparkle Pick-A-Size Paper Towels, White, 6 Dou...	3.853290e+18	1.0	6.44	96.43	2022-01-02	
1517230	345943601	0	7008027000	3-D Pet Products Premium Cockatiel Food, 4.5 l...	3.853290e+18	1.0	8.48	96.43	2022-01-02	
...	...	...	...	...	...	...	...	...	...	
2290480	290580467	0	2003006157	Beethoven / Beethoven's 2nd / Beethoven's 3rd ...	3.853290e+18	1.0	5.00	77.03	2023-12-31	
2290481	290580467	0	7000000000	Paw Patrol Aqua Pups (DVD) Home Page/Character...	3.853290e+18	1.0	5.00	77.03	2023-12-31	
2290482	290580467	0	999	The Smurfs / The Smurfs 2 / The Smurfs: The Le...	3.853290e+18	1.0	5.00	77.03	2023-12-31	
2290483	324284115	0	7009031000	Skeleton Skull Foot Pegs Black For Harley-Davi...	1.058424e+19	1.0	26.99	39.58	2023-12-31	
2290484	281008583	0	7000000000	Vintage Imperial Crown Cross Rhinestone Small ...	1.058424e+19	1.0	18.00	25.19	2023-12-31	

766260 rows × 15 columns

```
len(trans_df[(trans_df['event_date'] > '2022-01-01') & (trans_df['event_date'] < '2024-01-01')]['site_session_id'].unique())
```

```
1
```

Cleaning Session Dataframe

```
sesh_df = sesh_df[sesh_df['domain_name'] == 'ebay.com'].drop(['domain_name'], axis=1)
```

```
cleaned_sesh_df = sesh_df.copy()
```

```
sesh_df.shape
```

```
(2899219, 19)
```

```
cleaned_sesh_df.shape
```

```
(2899219, 19)
```

```
cleaned_sesh_df.columns
```

```
Index(['machine_id', 'site_session_id', 'user_session_id', 'domain_id',
       'ref_domain__name', 'pages_viewed', 'duration', 'event_date',
       'event_time', 'hoh_most_education', 'census_region', 'household_size',
       'hoh_oldest_age', 'household_income', 'children', 'racial_background',
       'connection_speed', 'hispanic', 'zip_code'],
      dtype='object')
```

TODO: find a way to deal with unknown values without just dropping them

```
# convert unknown values into null

# Define a dictionary mapping each column to a list of values to treat as unknown
unknown_values = {
    'racial_background': [99],
    'census_region': [99],
    'household_size': [-88, 99],
    'hoh_oldest_age': [99],
    'household_income': [99],
    'hoh_most_education': [-88, 99]
}

# Replace the specified values with np.nan in the DataFrame
cleaned_sesh_df.replace(unknown_values, np.nan, inplace=True)
```

```
from datetime import datetime

# convert event_time to python.time object instead of string
cleaned_sesh_df['event_time'] = cleaned_sesh_df['event_time'].apply(
    lambda x: datetime.strptime(x, "%H:%M:%S").time() if pd.notnull(x) and isinstance(x, str) else None
)
```

```
# missing percent
cleaned_sesh_df.isnull().mean() * 100
```

	0
machine_id	0.000000
site_session_id	0.000000
user_session_id	0.000000
domain_id	0.000000
ref_domain__name	75.844184
pages_viewed	0.203124
duration	0.228648
event_date	0.096957
event_time	0.151144
hoh_most_education	60.068211
census_region	0.703327
household_size	1.531792
hoh_oldest_age	0.496479
household_income	0.909866
children	0.098682
racial_background	0.625375
connection_speed	6.998471
hispanic	0.000000
zip_code	2.825071

dtype: float64

Note: `hoh_most_education` has around a 60% null percentage, which makes it somewhat unreasonable to impute the missing data.

```
# attempt to drop the hoh_most_education column
cleaned_sesh_df = cleaned_sesh_df.drop(['hoh_most_education'], axis=1)


# dropping null rows of columns with very small null percentages
# cleaned_sesh_df = cleaned_sesh_df.dropna(subset=['pages_viewed', 'duration', 'event_date', 'event_time', 'hoh_most_education', 'census_reg

cleaned_sesh_df = cleaned_sesh_df.dropna(subset=['pages_viewed', 'duration', 'event_date', 'event_time', 'census_region', 'hoh_oldest_age',


# 1. Replace missing ref_domain__name with "NoReferral"
cleaned_sesh_df['ref_domain__name'] = cleaned_sesh_df['ref_domain__name'].fillna('NoReferral')

# 2. Determine the top 10 referral domains (excluding "NoReferral")
domain_counts = cleaned_sesh_df.loc[cleaned_sesh_df['ref_domain__name'] != 'NoReferral', 'ref_domain__name'].value_counts()
top10_domains = domain_counts.head(10).index.tolist()

# 3. Create a new column that categorizes each referral appropriately
def categorize_ref_domain(domain):
    if domain == 'NoReferral':
        return 'NoReferral'
    elif domain in top10_domains:
        return domain
    else:
        return 'Others'

cleaned_sesh_df['ref_domain_category'] = cleaned_sesh_df['ref_domain__name'].apply(categorize_ref_domain)

# 4. One-hot encode the new column
# This will create dummy columns for each category: top 10 referral domains, 'NoReferral', and 'Others'
cleaned_sesh_df = pd.get_dummies(cleaned_sesh_df, columns=['ref_domain_category'], prefix='ref_domain')


cleaned_sesh_df.drop(columns=['ref_domain__name'], inplace=True)


cleaned_sesh_df
```

	machine_id	site_session_id	user_session_id	domain_id	pages_viewed	duration	event_date	event_time	census_region
0	288084937	6729633401426801739	2412880849370001	1.058424e+19	14.0	14.0	2020-01-01	00:00:02	3.0
1	288912140	1092751701181111355	2412889121400001	1.058424e+19	34.0	31.0	2020-01-01	00:00:02	1.0
7	274888709	5148178385048755113	2412748887090001	1.058424e+19	21.0	21.0	2020-01-01	00:00:04	3.0
11	282355046	6924161971643885156	2412823550460001	1.058424e+19	20.0	11.0	2020-01-01	00:00:09	1.0
12	297138715	3905965572138445730	2412971387150001	1.058424e+19	16.0	14.0	2020-01-01	00:00:09	4.0
...	...	...	...	...	...	...	...	...	...
13756382	302332442	1918747377201310 25	2533023324420009	1.058424e+19	30.0	51.0	1910-04-01	17:00:29	4.0
13758037	301312440	2733170316382653997	2623013124400033	1.058424e+19	12.0	5.0	2042-12-07	18:59:35	4.0
13759604	309738893	3315075966668160536	2573097388930055	1.058424e+19	1.0	1.0	1966-01-07	01:56:55	1.0
13761406	303646530	1884611728094424144	2833036465300155	1.058424e+19	1.0	1.0	2006-10-15	22:05:14	4.0
13765062	317127486	8446436198768662147	2883171274860058	1.058424e+19	2.0	1.0	1939-02-15	06:22:52	2.0

2798922 rows × 29 columns

```
cleaned_sesh_df = cleaned_sesh_df.reset_index(drop=True)

# imputing household_size, connection_speed, zip_code, hoh_most_education based on census_region and their respective distributions
def impute_by_region_mode(group, col):
  mode_val = group[col].mode()

  if not mode_val.empty:
    group[col].fillna(mode_val[0], inplace=True)

  return group

def impute_by_region_median(group, col):
  median_val = group[col].median()

  group[col].fillna(median_val, inplace=True)
```

```
  return group
```

```
cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_mode(g, 'connection_speed')).r

cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_median(g, 'household_size'))

cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_mode(g, 'zip_code'))

# cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_mode(g, 'hoh_most_education'
```

```
<ipython-input-14-5ce7a25d187d>:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignm
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me


  group[col].fillna(mode_val[0], inplace=True)
<ipython-input-14-5ce7a25d187d>:21: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprec
  cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_mode(g, 'connection_spee
<ipython-input-14-5ce7a25d187d>:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me


  group[col].fillna(median_val, inplace=True)
<ipython-input-14-5ce7a25d187d>:23: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprec
  cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_median(g, 'household_siz
<ipython-input-14-5ce7a25d187d>:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignm
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me


  group[col].fillna(mode_val[0], inplace=True)
<ipython-input-14-5ce7a25d187d>:25: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprec
  cleaned_sesh_df = cleaned_sesh_df.groupby('census_region', group_keys=False).apply(lambda g: impute_by_region_mode(g, 'zip_code'))
```

Creating new features

```
cleaned_sesh_df
```

	machine_id	site_session_id	user_session_id	domain_id	pages_viewed	duration	event_date	event_time	census_region
0	288084937	6729633401426801739	2412880849370001	1.058424e+19	14.0	14.0	2020-01-01	00:00:02	3.0
1	288912140	1092751701181111355	2412889121400001	1.058424e+19	34.0	31.0	2020-01-01	00:00:02	1.0
2	274888709	5148178385048755113	2412748887090001	1.058424e+19	21.0	21.0	2020-01-01	00:00:04	3.0
3	282355046	6924161971643885156	2412823550460001	1.058424e+19	20.0	11.0	2020-01-01	00:00:09	1.0
4	297138715	3905965572138445730	2412971387150001	1.058424e+19	16.0	14.0	2020-01-01	00:00:09	4.0
...	...	...	...	...	...	...	...	...	...
2798917	302332442	191874737720131025	2533023324420009	1.058424e+19	30.0	51.0	1910-04-01	17:00:29	4.0
2798918	301312440	2733170316382653997	2623013124400033	1.058424e+19	12.0	5.0	2042-12-07	18:59:35	4.0
2798919	309738893	3315075966668160536	2573097388930055	1.058424e+19	1.0	1.0	1966-01-07	01:56:55	1.0
2798920	303646530	1884611728094424144	2833036465300155	1.058424e+19	1.0	1.0	2006-10-15	22:05:14	4.0
2798921	317127486	8446436198768662147	2883171274860058	1.058424e+19	2.0	1.0	1939-02-15	06:22:52	2.0

2798922 rows × 29 columns

```
# combine event_date and event_time
cleaned_sesh_df['transaction_datetime'] = pd.to_datetime(
    cleaned_sesh_df['event_date'].astype(str) + ' ' + cleaned_sesh_df['event_time'].astype(str)
)

cleaned_sesh_df = cleaned_sesh_df.drop(['event_date', 'event_time'], axis=1)

# note that we are not creating the indicator features relating to date-time because those are already in trans_df


# calculating the "attention" for each page
cleaned_sesh_df['avg_time_second_per_page'] = (cleaned_sesh_df['duration'] * 60 )/ cleaned_sesh_df['pages_viewed']

cleaned_sesh_df['pages_per_second'] = cleaned_sesh_df['pages_viewed'] / (cleaned_sesh_df['duration'] * 60) # duration is given in minutes


cleaned_sesh_df.to_csv("cleaned_sesh_df.csv", index=False)
```

Plotting Data

```
cleaned_sesh_df['machine_id'].describe()
```

	machine_id
count	2.798922e+06
mean	3.054333e+08
std	2.522721e+07
min	5.438873e+07
25%	2.886735e+08
50%	3.056827e+08
75%	3.227516e+08
max	3.715759e+08

dtype: float64

```
cleaned_sesh_df['machine_id'].value_counts().describe()
```

	count
count	101426.000000
mean	27.595705
std	143.800667
min	1.000000
25%	1.000000
50%	3.000000
75%	10.000000
max	7222.000000

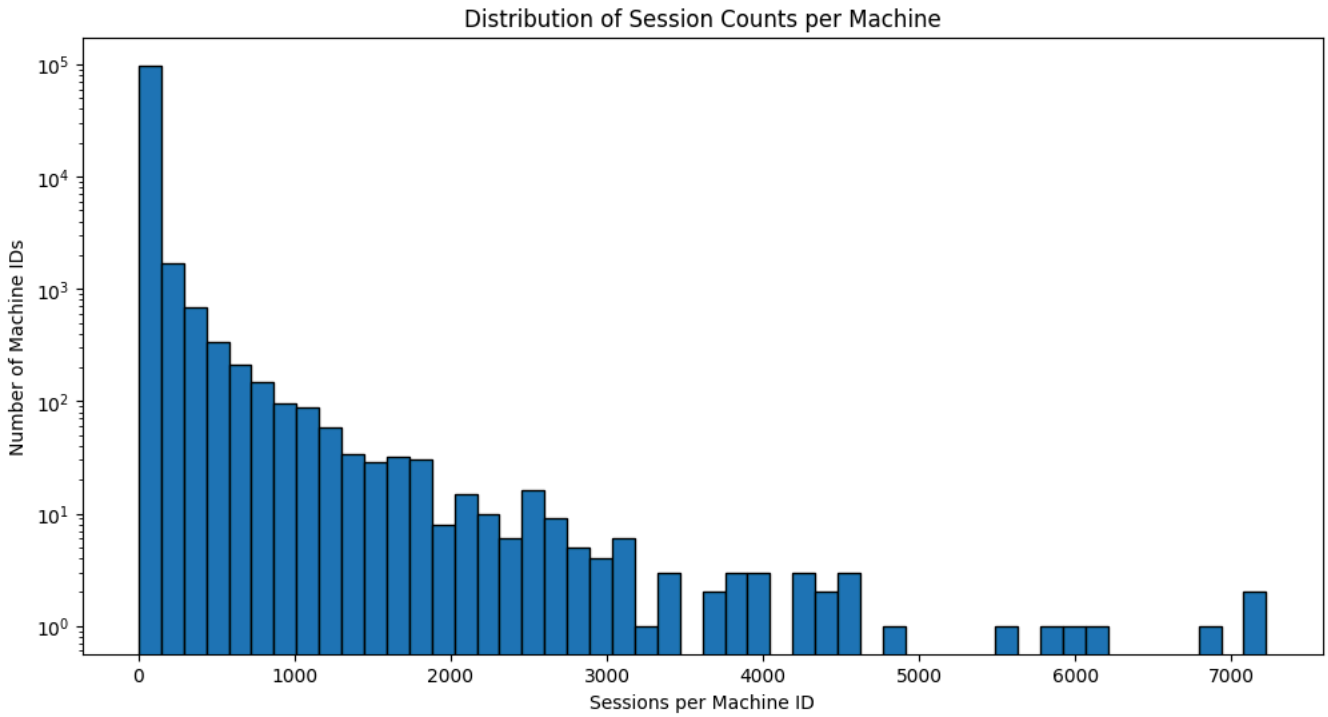
dtype: float64

```
import matplotlib.pyplot as plt

# Calculate the frequency count for each machine_id
machine_counts = cleaned_sesh_df['machine_id'].value_counts()

# Plot histogram of these frequencies
plt.figure(figsize=(12,6))
plt.hist(machine_counts, bins=50, edgecolor='black')
plt.xlabel('Sessions per Machine ID')
plt.ylabel('Number of Machine IDs')
plt.title('Distribution of Session Counts per Machine')

# Because the distribution is highly skewed, use a log scale on the y-axis:
plt.yscale('log')
plt.show()
```




```
cleaned_sesh_df.columns
```

```
Index(['machine_id', 'site_session_id', 'user_session_id', 'domain_id',
       'pages_viewed', 'duration', 'census_region', 'household_size',
       'hoh_oldest_age', 'household_income', 'children', 'racial_background',
       'connection_speed', 'hispanic', 'zip_code', 'ref_domain_NoReferral',
       'ref_domain_Others', 'ref_domain_bing.com', 'ref_domain_duckduckgo.com',
       'ref_domain_ebay.com', 'ref_domain_facebook.com',
       'ref_domain_google.com', 'ref_domain_googlesyndication.com',
       'ref_domain_msn.com', 'ref_domain_redbrain.shop',
       'ref_domain_yahoo.com', 'ref_domain_youtube.com',
       'transaction_datetime', 'avg_time_second_per_page', 'pages_per_second'],
      dtype='object')
```

Cleaning Transaction Dataframe

```
trans_df
```

	machine_id	site_session_id	prod_category_id	prod_name	domain_id	prod_qty	prod_totprice	basket_tot	event_date	ev
0	285811167	7837313750729559995	1001001014	SaiLiiny Vest for Boys 3 Buttons Slim Fit Dres...	4.046670e+18	1.0	15.66	593.07	2020-01-01	
1	285811167	7837313750729559995	1002004139	Mr and Mrs Cake Topper Rhinestone Crystal Meta...	4.046670e+18	1.0	17.99	593.07	2020-01-01	
2	285811167	7837313750729559995	4005017229	Hoffmaster 114000 Plastic Tablecover Roll, 300...	4.046670e+18	2.0	38.24	593.07	2020-01-01	
3	285811167	7837313750729559995	1002004139	HiiARug Handmade Wedding Bouquets, Artificial ...	4.046670e+18	1.0	35.98	593.07	2020-01-01	
4	285811167	7837313750729559995	1001001014	CHAMA Men's Formal Classic Fit Business Dress ...	4.046670e+18	1.0	24.69	593.07	2020-01-01	
...	...	...	...	...	...	...	...	...	...	
2308967	319230819	0	7009032145	Decorative Kitchen Dish and Bath Hand Towels ...	4.046670e+18	1.0	0.00	0.00	2016-05-07	
2308968	286012823	0	7008024095	Bob Evans Gluten-Free Original Mashed Potatoes...	3.853290e+18	1.0	3.47	124.15	NaN	
2308969	290757481	0	7009031113	RK ROYAL KLUDGE RK84 Wireless RGB 75 Triple M...	4.046670e+18	1.0	39.86	198.66	NaN	
2308970	260995801	0	999	LILLUSORY Womens Turtleneck	4.046670e+18	1.0	10.95	16.21	NaN	
2308971	363299409	0	7009031127		4.046670e+18	1.0	18.99	131.96	NaN	

```
cleaned_trans_df = trans_df.copy()
```

```
cleaned_trans_df = cleaned_trans_df[cleaned_trans_df['domain_name'] == 'ebay.com'].drop(['domain_name'], axis=1)
```

```
cleaned_trans_df
```

2308972 rows × 15 columns

	machine_id	site_session_id	prod_category_id	prod_name	domain_id	prod_qty	prod_totprice	basket_tot	event_date
20	300908044	6068113588223141415	3004000000	2Pcs Replacement Ear Pads Cup Cushion for Beat...	1.058424e+19	1.0	8.98	9.78	2020-01-01
33	300908044	6068113588223141415	3004010078	2pcs Replacement Leather Ear Pads Cushion For ...	1.058424e+19	2.0	12.76	8.75	2020-01-01
47	300908044	6068113588223141415	3004000000	2Pcs Replacement Ear Pads Cup Cushion for Beat...	1.058424e+19	1.0	4.67	5.08	2020-01-01
51	286745732	8467790125069013688	5006019207	1984 FORD LTD OWNER GUIDE MANUAL AUTOMOBILE CA...	1.058424e+19	1.0	4.99	0.00	2020-01-01
56	290603168	8720786549804594449	1002003044	VINTAGE WOOL SCARF- Floral Print Folkloric Fou...	1.058424e+19	1.0	15.00	18.78	2020-01-01
...	...	...	...	...	...	...	...	...	...
2308933	290811361	6545131343187325434	7010037057	3 PCS OxyLED Motion Sensor Closet Lights Cordl...	1.058424e+19	1.0	14.99	21.36	NaN
2308935	306122371	0	7009032145	3x4" Clear Reclosable Bags 4-Mil Heavy-Duty Se...	1.058424e+19	1.0	10.50	11.08	NaN
2308953	321469887	696933178624022619	7009031118	96" W x 72" L Porch Deck Roller Blind Sun Shad...	1.058424e+19	1.0	48.19	51.56	NaN
2308962	312286716	0	999	1973 Topps #556 Yankees Team EX/EX Yankees 53...	1.058424e+19	1.0	9.87	11.20	NaN
2308963	285223966	0	7000000000	The Guns of Navarone (DVD, 2000	1.058424e+19	1.0	1.14	4.71	NaN

```
# missing percent
cleaned_trans_df.isnull().mean() * 100
```

	0
machine_id	0.000000
site_session_id	0.000000
prod_category_id	0.000000
prod_name	0.000000
domain_id	0.000000
prod_qty	0.400457
prod_totprice	0.000000
basket_tot	1.204932
event_date	0.805809
event_time	0.117468
total_transactions_2020	43.289001
total_transactions_2021	34.360136
total_transactions_2022	57.538611
total_transactions_2023	72.830744

dtype: float64

```
# dropping site_session_id, prod_name, total_transactions_XXXX and remove null rows
# site_session_id is faulty... has a bunch of 0

cleaned_trans_df = cleaned_trans_df.drop(['site_session_id', 'prod_name', 'total_transactions_2020', 'total_transactions_2021', 'total_trans
cleaned_trans_df = cleaned_trans_df.dropna()
```

```
from datetime import datetime

# convert event_time to python.time object instead of string
cleaned_trans_df['event_time'] = cleaned_trans_df['event_time'].apply(
    lambda x: datetime.strptime(x, "%H:%M:%S").time() if pd.notnull(x) and isinstance(x, str) else None
)
```

```
cleaned_trans_df
```

	machine_id	prod_category_id	domain_id	prod_qty	prod_totprice	basket_tot	event_date	event_time
20	300908044	3004000000	1.058424e+19	1.0	8.98	9.78	2020-01-01	00:12:58
33	300908044	3004010078	1.058424e+19	2.0	12.76	21.51	2020-01-01	00:18:02
47	300908044	3004000000	1.058424e+19	1.0	4.67	5.08	2020-01-01	00:28:00
51	286745732	5006019207	1.058424e+19	1.0	4.99	4.99	2020-01-01	00:37:00
56	290603168	1002003044	1.058424e+19	1.0	15.00	18.78	2020-01-01	00:40:23
...	...	...	...	...	...	...	...	...
2305875	258467115	1001001010	1.058424e+19	1.0	99.88	149.83	2009-07-10	02:35:05
2306178	270808766	7009000000	1.058424e+19	1.0	46.87	49.73	2025-07-10	02:53:50
2306202	295618476	1002004139	1.058424e+19	1.0	41.67	44.17	2011-03-13	02:55:12
2306974	354180802	7009032145	1.058424e+19	1.0	11.99	12.71	2007-06-27	03:47:44
2307007	311505244	7009029022	1.058424e+19	1.0	13.98	15.14	2009-06-26	03:50:06

219118 rows × 8 columns

Creating new features

```
# combine event_date and event_time
cleaned_trans_df['transaction_datetime'] = pd.to_datetime(
    cleaned_trans_df['event_date'].astype(str) + ' ' + cleaned_trans_df['event_time'].astype(str
)
```

```
# generating date time related features
cleaned_trans_df['transaction_hour'] = cleaned_trans_df['transaction_datetime'].dt.hour
cleaned_trans_df['transaction_dayofweek'] = cleaned_trans_df['transaction_datetime'].dt.dayofweek  # Monday=0, Sunday=6
cleaned_trans_df['transaction_month'] = cleaned_trans_df['transaction_datetime'].dt.month

cleaned_trans_df['is_weekend'] = cleaned_trans_df['transaction_dayofweek'].isin([5, 6]).astype(int)


# potential proxy to see if a single product is dominating the basket
cleaned_trans_df['basket_vs_product_diff'] = cleaned_trans_df['basket_tot'] - cleaned_trans_df['prod_totprice']
```

```
cleaned_trans_df
```

	machine_id	prod_category_id	domain_id	prod_qty	prod_totprice	basket_tot	transaction_datetime	transaction_hour	transac
20	300908044	3004000000	1.058424e+19	1.0	8.98	9.78	2020-01-01 00:12:58	0	
33	300908044	3004010078	1.058424e+19	2.0	12.76	21.51	2020-01-01 00:18:02	0	
47	300908044	3004000000	1.058424e+19	1.0	4.67	5.08	2020-01-01 00:28:00	0	
51	286745732	5006019207	1.058424e+19	1.0	4.99	4.99	2020-01-01 00:37:00	0	
56	290603168	1002003044	1.058424e+19	1.0	15.00	18.78	2020-01-01 00:40:23	0	
...	...	...	...	...	...	...	...	...	
2305875	258467115	1001001010	1.058424e+19	1.0	99.88	149.83	2009-07-10 02:35:05	2	
2306178	270808766	7009000000	1.058424e+19	1.0	46.87	49.73	2025-07-10 02:53:50	2	
2306202	295618476	1002004139	1.058424e+19	1.0	41.67	44.17	2011-03-13 02:55:12	2	
2306974	354180802	7009032145	1.058424e+19	1.0	11.99	12.71	2007-06-27 03:47:44	3	
2307007	311505244	7009029022	1.058424e+19	1.0	13.98	15.14	2009-06-26 03:50:06	3	

219118 rows × 12 columns

```
cleaned_trans_df[cleaned_trans_df['basket_vs_product_diff'] < 0]
```

	machine_id	prod_category_id	domain_id	prod_qty	prod_totprice	basket_tot	transaction_datetime	transaction_hour	transaction_dayo

```
cleaned_trans_df
```

	machine_id	prod_category_id	domain_id	prod_qty	prod_totprice	basket_tot	transaction_datetime	transaction_hour	transac
20	300908044	3004000000	1.058424e+19	1.0	8.98	9.78	2020-01-01 00:12:58	0	
33	300908044	3004010078	1.058424e+19	2.0	12.76	21.51	2020-01-01 00:18:02	0	
47	300908044	3004000000	1.058424e+19	1.0	4.67	5.08	2020-01-01 00:28:00	0	
51	286745732	5006019207	1.058424e+19	1.0	4.99	4.99	2020-01-01 00:37:00	0	
56	290603168	1002003044	1.058424e+19	1.0	15.00	18.78	2020-01-01 00:40:23	0	
...	...	...	...	...	...	...	...	...	
2305875	258467115	1001001010	1.058424e+19	1.0	99.88	149.83	2009-07-10 02:35:05	2	
2306178	270808766	7009000000	1.058424e+19	1.0	46.87	49.73	2025-07-10 02:53:50	2	
2306202	295618476	1002004139	1.058424e+19	1.0	41.67	44.17	2011-03-13 02:55:12	2	
2306974	354180802	7009032145	1.058424e+19	1.0	11.99	12.71	2007-06-27 03:47:44	3	
2307007	311505244	7009029022	1.058424e+19	1.0	13.98	15.14	2009-06-26 03:50:06	3	

219118 rows × 12 columns

```
cleaned_trans_df['machine_id'].describe()
```

machine_id